Міністерство освіти і науки України
Харківський національний університет радіоелектроніки

Кафедра штучного інтелекту

Дисципліна «Аналітика великих даних»

Практичне заняття №3

«Потокова обробка великих даних засобами Apache Kafka та Spark Structured Streaming»

Виконав:
ст. гр. ДМм-26-1
Зелінський В.С.

Прийняв:
Золотухін О.В.

Харків – 2026

Потокова обробка великих даних засобами Apache Kafka та Spark Structured Streaming

Мета роботи

Ознайомитися з принципами потокової обробки великих даних та архітектурою Apache Kafka і Spark Structured Streaming; отримати практичні навички формування потоку даних, передавання повідомлень через Kafka та їх обробки засобами Spark у режимі, наближеному до реального часу, на прикладі хвилинних свічок BTC/USD (Bitcoin Historical Data, 7 816 537 записів за 2012–2026 роки).

Короткі теоретичні відомості

У роботі кожна хвилинна свічка – окрема подія потоку. Kafka зберігає повідомлення (ключ, значення, мітка часу) у темі, поділеній на партиції; партицію визначає хеш ключа, а порядок гарантується лише всередині партиції. Тому ключ обрано так, щоб хвилини одного періоду потрапляли в одну партицію. Spark Structured Streaming виконує план DataFrame над темою мікробатчами за тригером, а checkpoint зберігає прочитані зміщення та стан агрегацій.

Tumbling-вікна мають фіксовану довжину й не перетинаються; вікно визначається за часом події (поле `ts` свічки), а не за часом обробки. Водяна позначка дорівнює максимальному баченому часу події мінус допустиме запізнення; стан старших вікон видаляється, пізні записи відкидаються. Позначка одна на весь запит, а не на партицію, що важливо, коли партиції містять різні роки. Режим append записує лише закриті позначкою вікна, update – змінені рядки.

Результати виконання практичного заняття

1 Підготовка Kafka

Kafka 3.7.0 запущено як один вузол KRaft із ролями `broker,controller` (старт за 10,1 с); Spark працює в режимі `local[2]`, щоб частина ядер лишалася брокеру та producer (таблиця 1).

Таблиця 1 – Програмне середовище та параметри запуску

Компонент або параметр	Значення
Машина	4 vCPU (спільні з іншими завданнями), 15,7 ГіБ RAM, Ubuntu 24.04.5 LTS
Apache Kafka / Java	3.7.0 (Scala 2.13), KRaft, 1 брокер / OpenJDK 21.0.12.1
Spark / PySpark / конектор	4.3.0 / 4.3.0 / spark-sql-kafka-0-10_2.13:4.3.0
Producer	confluent-kafka 2.16.1 (librdkafka), acks=all, enable.idempotence=true
Python / pandas / pyarrow / seed	3.13.18 / 3.0.5 / 26.0.1 / 42
spark.master / driver.memory / shuffle.partitions	local[2] / 2g / 4
Часова зона сесії / тригер / maxOffsetsPerTrigger	UTC / 10 с / 3000

Тему `bigdata-topic` створено з 3 партиціями та фактором реплікації 1 (рисунок 1).

```
$ kafka-topics.sh --bootstrap-server localhost:9092 --create --topic bigdata-topic --parti
  tions 3 --replication-factor 1 && kafka-topics.sh --bootstrap-server localhost:9092 --de
  scribe --topic bigdata-topic
Created topic bigdata-topic.
Topic: bigdata-topic    TopicId: ubNpjfNGQ9aCN1eQZIYtbg    PartitionCount: 3    Replicatio
↪ nFactor: 1    Configs: segment.bytes=1073741824
    Topic: bigdata-topic    Partition: 0    Leader: 1    Replicas: 1    Isr: 1
    Topic: bigdata-topic    Partition: 1    Leader: 1    Replicas: 1    Isr: 1
    Topic: bigdata-topic    Partition: 2    Leader: 1    Replicas: 1    Isr: 1
```

Рисунок 1 – Створення та опис теми bigdata-topic

Ключ повідомлення – рік-місяць UTC, тому порядок хвилин кожного періоду зберігається. Кінцеві зміщення партицій 10 080 / 20 160 / 0: ключі `2020-03` і `2021-05` дали однаковий залишок хешу librdkafka, що не впливає на коректність.

2 Формування потоку

Producer (`producer.py`) надсилає хвилини трьох періодів обвалів (таблиця 2) хронологічно, одне JSON-повідомлення на хвилину; ідемпотентний режим виключає дублювання й перестановку.

```
key = time.strftime("%Y-%m", time.gmtime(ts))   # ключ = період
val = {"ts": ts, "open": num(row.Open), "high": num(row.High), "low":
   num(row.Low),
        "close": num(row.Close), "volume": num(row.Volume)}
prod.produce(a.topic, key=key, value=json.dumps(val), on_delivery=on_delivery)
prod.poll(0)
time.sleep(a.delay_ms / 1000.0)                  # затримка 10 мс
```

Таблиця 2 – Відтворювані періоди

Ключ	Дати (UTC)	Хвилин	Подія	Денна лог-дохідність	Партиція
2020-03	09–16.03.2020	11 520	обвал COVID-19, 12–13.03	−0,494 / +0,150	1
2021-05	17–22.05.2021	8 640	обвал 19.05	−0,153	1
2022-11	06–12.11.2022	10 080	крах FTX, 08–09.11	−0,104 / −0,156	0

Затримка 10 мс прискорює потік: 10 хв ринкового часу відтворюються приблизно за 0,1 с (рисунок 2).

```
$ sed -n '1,5p;$p' run/logs/producer.log | cut -c1-300
[producer] рядків до відправки: 30240; затримка 10.0 мс; тема bigdata-topic
%4|1794438768.553|GETPID|rdkafka#producer-1| [thrd:main]: Failed to acquire idempotence PI
↪ D from broker localhost:9092/1: Broker: Coordinator load in progress: retrying
[producer] надіслано 2000/30240  (87.7 повід./с)
[producer] надіслано 4000/30240  (87.2 повід./с)
[producer] надіслано 6000/30240  (88.7 повід./с)
[producer] завершено: {"sent": 30240, "delivery_errors": 0, "seconds": 322.093, "msg_per_s
↪ ": 93.886, "delay_ms": 10.0, "topic": "bigdata-topic", "periods": "2020-03-09:2020-03-16
↪ ,2021-05-17:2021-05-22,2022-11-06:2022-11-12"}
```

Рисунок 2 – Журнал producer: 30 240 повідомлень без помилок доставки

Надіслано 30 240 повідомлень за 322,093 с (93,886 повід./с), помилок доставки 0; перечитування групою `pz3-verify` дало 30 240 повідомлень з нульовим lag.

3 Отримання потоку

Застосунок `streaming_job.py` читає тему через конектор `spark-sql-kafka-0-10`:

```
raw = (spark.readStream.format("kafka")
       .option("kafka.bootstrap.servers", a.bootstrap)
       .option("subscribe", a.topic)
       .option("startingOffsets", "earliest")
       .option("maxOffsetsPerTrigger", a.max_offsets)
       .option("failOnDataLoss", "true")
       .load())
```

Після першого запуску зміщення беруться з checkpoint; кожен із семи запитів читає тему окремо. Мікробатчі містили 11–977 рядків (медіана 949), ліміт 3000 не досягався. На 22-му мікробатчі (рисунок 3) усі запити мають статус RUNNING, вхід 85,75–87,56 рядків/с, обробка 314,64–814,77 рядків/с.

Name	Status	Duration	Avg Input /sec	Avg Process /sec	Latest Batch
anomaly	RUNNING	3 minutes 50 seconds	85.75	814.77	22
console_1h	RUNNING	3 minutes 50 seconds	85.89	386.14	22
daily_mem	RUNNING	3 minutes 51 seconds	87.56	444.53	22
win1h_mem	RUNNING	3 minutes 51 seconds	86.05	443.91	22
win15m_mem	RUNNING	3 minutes 52 seconds	85.94	466.19	22
sink_win1h	RUNNING	3 minutes 52 seconds	86.75	350.43	22
sink_win15m	RUNNING	3 minutes 53 seconds	87.23	314.64	22

Рисунок 3 – Сім активних потокових запитів одного застосунку у Spark UI (Structured Streaming, мікробатч 22; фрагмент таблиці: колонки ID, Run ID і Start Time пропущено)

4 Обробка потоку

Ключ і значення приводяться з `binary` до `string`, JSON розбирається `from_json` за явною схемою (`ts` – `LongType`, п'ять полів `DoubleType`). Фільтр відкидає порожні поля, `high < low`, `close ≤ 0` і від'ємний обсяг (таких 0 з 30 240). Групування й агрегування виконує запит `daily_mem`: добове tumbling-вікно з кількістю хвилин, середнім і стандартним відхиленням `close`, мінімумом, максимумом і сумою обсягу; результат – 21 доба по 1 440 хвилин. Console-приймач (update) для мікробатчу 12 вивів 1-год вікна 15.03.2020 (рисунок 4).

```
$ grep -a -A 7 -m1 '^Batch: 12$' run/logs/spark_job.log
Batch: 12
-------------------------------------------
+-------------------+-------------------+-------+-------+-------+-------+-----------------
↪ -+---------+
|win_start          |win_end            |open   |high   |low    |close  |volume
↪  |n_candles|
+-------------------+-------------------+-------+-------+-------+-------+-----------------
↪ -+---------+
|2020-03-15 12:00:00|2020-03-15 13:00:00|5437.45|5585.11|5266.32|5301.05|1600.241103210000
↪ 3|60       |
|2020-03-15 13:00:00|2020-03-15 14:00:00|5300.52|5362.02|5295.5 |5327.36|239.8456579000000
↪ 2|60       |
|2020-03-15 16:00:00|2020-03-15 17:00:00|5313.45|5336.97|5250.9 |5285.62|375.1527277999998
↪ 7|60       |
```

Рисунок 4 – Вивід console-приймача: віконні агрегати мікробатчу 12

5 Віконна обробка

5.1 Віконні агрегати

Вікна коротші за 1 хв містили б одну свічку, тому обрано tumbling-вікна 15 хв (динаміка обвалу всередині години) і 1 год (масштаб П31–П32):

```
events = valid.withWatermark("event_time", "10 minutes")
g = events.groupBy(F.window("event_time", "1 hour")).agg(
    F.min(F.struct("event_time", "open")).getField("open").alias("open"),
    F.max("high").alias("high"), F.min("low").alias("low"),
    F.max(F.struct("event_time", "close")).getField("close").alias("close"),
    F.sum("volume").alias("volume"), F.count(F.lit(1)).alias("n_candles"))
```

Open і close взято через `min/max` структури за `event_time`, бо `first/last` у потоці залежать від порядку рядків у мікробатчі. Результат перевірено скриптом `verify_windows.py` проти `pandas.resample` (таблиця 3).

Таблиця 3 – Порівняння віконних агрегатів Spark і pandas

Показник	Вікна 15 хв	Вікна 1 год
Вікон у pandas	2 016	504
Рядків memory-приймача (update)	2 048	536
Вікон Spark (останнє оновлення)	2 016	504
Вікон лише в одному джерелі	0	0
max |Δ| OHLC, n_candles	0	0
max |Δ| volume	$4{,}5\cdot10^{-13}$	$2{,}7\cdot10^{-12}$
Вікон у parquet (append), дублікатів	2 015, 0	503, 0
Не записано в parquet	1	1

Розбіжність обсягу порядку 10^{-12} – наслідок порядку додавання чисел з плаваючою комою. Parquet-приймач не записав останнє вікно кожного масштабу: вікно 23:00–24:00 12.11.2022 закрилося б лише подією не раніше 00:10 13.11.2022, а потік закінчується о 23:59. На 15-хв вікнах (рисунок 5) видно дві хвилі обвалу 12.03.2020 (близько 10:30–11:30 UTC і наприкінці доби) з обсягом 1-год вікон 8 925 BTC (11:00) і 8 051 BTC (23:00).



Рисунок 5 – OHLC за 15-хв вікнами (12.03.2020) і 1-год вікнами (9–16.03.2020)

5.2 Водяна позначка

Невпорядкованість подій – 0 з 30 240 як у партиціях, так і в порядку надходження, `numRowsDroppedByWatermark` = 0 в усіх запитах. Отже, 10 хв – запас для живого джерела, де свічка може запізнитися через перепідключення до біржі; ціна запасу – append-результат на 10 хв часу подій пізніше (≈ 0,1 с при відтворенні). Остання позначка – 2022-11-12 23:49. «Global Watermark Gap» у Spark UI змінюється сходинками від $2{,}107 \cdot 10^8$ с (2020 рік) до $1{,}261 \cdot 10^8$ с (2022 рік), медіана $1{,}730 \cdot 10^8$ с ≈ 5,48 року; це вік історичних подій, а не затримка обробки.

Через глобальність позначки (`multipleWatermarkPolicy = min` діє лише між кількома вхідними позначками) виконано експеримент backlog: запит вікон 15 хв читав заздалегідь заповнену тему з `maxOffsetsPerTrigger = 3000`, партиція 0 (листопад 2022 року) одразу просунула позначку, і за 12 мікробатчів збережено лише 14 080 з 30 240

рядків (939 з 2 016 вікон). `numRowsDroppedByWatermark` показала лише 1 083 рядки – близько до кількості втрачених вікон (1 077), тобто лічильник, імовірно, рахує частково агреговані рядки, але це окремо не перевірялося. Без ліміту (один мікробатч) збережено всі 30 240 рядків; отже, періоди різних епох у різних партиціях безпечні лише за послідовного надсилання і читання наживо.

6 Виявлення аномалій

6.1 Перша версія правила

Правило v1 ($|r| > k \cdot 1{,}4826 \cdot \mathrm{MAD}$ за 60 хв або обсяг у k_v разів вищий за 60-хв медіану) за $k \in \{3, 5, 7, 10\}$ позначало 9,9–27,1 % хвилин, а в періодах обвалів – лише 0,53–0,90 від звичайної частоти: 60-хв база встигала підлаштуватися під обвал.

6.2 Правило v2

Нехай $r_t = \ln(C_t / C_{t-1})$. Хвилини з нульовим обсягом (16,79 % історії) – заповнення сітки, тому не оцінюються і не входять до бази. База B_t – активні ($V > 0$) хвилини серед 10 080 попередніх (7 діб):

```
sigma_t = Q0,90{|r_i| : i ∈ B_t} / 1,6449
z_ret,t = |r_t| / sigma_t
z_vol,t = (ln V_t − Q0,50{ln V_i}) / ((Q0,75{ln V_i} − Q0,25{ln V_i}) / 1,349)
аномалія_t ⇔ z_ret,t > k і z_vol,t > 0,   k = 9
```

Константи 1,6449 і 1,349 узгоджують шкали з σ нормального розподілу; база має містити не менше 3 360 активних хвилин у базі, тому оцінено 6 453 867 з 7 816 537 хвилин.

Формулу обрано серед 19 варіантів за однакового бюджету 0,10 % позначених активних хвилин 2018+ за lift (частка позначених хвилин у п'яти днях подій, поділена на частку в інші дні 2018+), частотою позначок

2012–2016 років і рангами днів подій (таблиця 4; проєктна вибірка, оцінка поза нею – у підрозділі 6.5).

Таблиця 4 – Варіанти правила за однакового бюджету тривог 0,10 % (2018+), проєктна вибірка

База	Шкала або комбінація	Поріг	Lift	Позначено хвилин 2012–2016, %	Ранги 12.03.20 / 13.03.20 / 19.05.21 / 08.11.22 / 09.11.22
60 хв	MAD	118,11	0,0	1,7518	1286 / 1286 / 1286 / 1286 / 1286
24 год	MAD	37,68	0,3	0,9245	352 / 482 / 482 / 352 / 482
24 год	Q90	8,64	7,0	0,0499	9 / 1895 / 20 / 35 / 1895
7 діб	MAD	13,14	36,9	3,4019	258 / 205 / 367 / 231 / 303
7 діб	Q90	8,87	38,4	0,0648	1 / 2 / 25 / 4 / 65
7 діб	Q99	5,72	17,2	0,0604	3 / 34 / 12 / 11 / 434
7 діб	лише обсяг	2,59	11,3	0,5099	381 / 1133 / 25 / 219 / 1787
7 діб	Q90 і z_vol > 0	8,87	38,4	0,0613	1 / 2 / 25 / 4 / 63
60 хв (v1)	MAD або обсяг	1,28	0,3	5,0698	3406 / 3406 / 3406 / 2262 / 3406

За базою 60 хв і 24 год шкала зростає вже під час обвалу (lift не більше 7,0); 7-денна база охоплює тижневий цикл і реагує повільніше (lift 36,9–38,4). Q90 обрано замість MAD через точні нулі дохідності: частка активних хвилин з r = 0 – 42,6 % у 2012 році й 29,4 % у 2013 році проти 0,4 % у 2021 році, тому за MAD 2012–2016 роки отримують 3,40 % позначок, а за Q90 – 0,0648 %. Обсяг слабкий як тригер (99,99-й перцентиль z_vol у 2018+ лише 3,067; правило лише за обсягом – lift 11,3 і 60,6 % позначок у 2012–2016 роках), тому він лише підтверджує: z_vol > 0 дає 5 437 позначок замість 5 489 без втрати lift, а z_vol > 2 знижує lift до 23,2.

6.3 Вибір порогу k

Поріг – найменше k із сітки, за якого позначається не більше 0,10 % активних хвилин 2018+, тобто близько 1,4 тривоги на добу для ручного перегляду (таблиця 5).

Таблиця 5 – Кількість спрацювань правила v2 для різних k (уся історія)

k	Позначено хвилин	Частка 2018+, %	Тривог на добу 2018+	Частка 2012–2016, %	Хвилин у днях подій	Lift (хвилини)	Lift (вікна 1 год)
3	126 956	2,1178	30,50	1,5564	1 636	10,9	2,6
5	29 528	0,5093	7,33	0,3318	761	21,4	5,7
7	11 009	0,1918	2,76	0,1213	398	30,2	8,5
8	7 527	0,1321	1,90	0,0816	307	34,0	9,7
9	5 437	0,0959	1,38	0,0582	251	38,5	11,8
10	3 985	0,0702	1,01	0,0431	199	41,9	13,0
20	540	0,0095	0,14	0,0062	39	62,4	33,6

За k = 8 частка ще 0,1321 %, тому обрано k = 9 (0,0959 %, 1,38 тривоги на добу). Більше k підвищує lift, але зменшує кількість позначених хвилин у днях подій (251 за k = 9, 39 за k = 20).

6.4 Потокова реалізація

У потоці правило виконує `foreachBatch` з класом `StreamScorer` (`anomaly_rule.py`), що для кожного ключа Kafka тримає буфер останніх 10 081 хвилини. Тижневу базу неможливо накопичити з 6–8-денних періодів, тому буфер прогрівається 10 081 хвилиною історії з parquet. Найбільше аномалій дали мікробатчі 8 (95; ніч 12–13.03.2020), 7 (56; 12.03.2020) і 27 (56; 08.11.2022); мікробатч 18 містить 23 аномалії 19.05.2021.

6.5 Оцінка поза вибіркою

Числа таблиці 4 оптимістичні, тому виконано дві перевірки (рисунок 6). У leave-one-event-out варіант обирається за lift на чотирьох днях і оцінюється на п'ятому: 7-денна база обирається в усіх п'яти фолдах, шкала – Q90 у трьох, IQR(r) в одному, «та сама година» в одному; середній lift пропущеного дня 30,7 проти 38,4, тобто проєктна оцінка завищена приблизно на п'яту частину. У часовому поділі «подіями» навчання були 118 днів 2013–2019 років з |денна лог-дохідність| ≥ 0,10, на них же

підбирався k. Найкращим виявився варіант 7 діб, Q90 (k = 8,38): на 2020+ він дає lift 42,5 для п'яти відомих днів і 0,094 % позначок у базові дні, тобто бюджет переноситься на нові роки. Обмеження: «події» навчання – не розмічені аномалії, а дні великої зміни ціни.

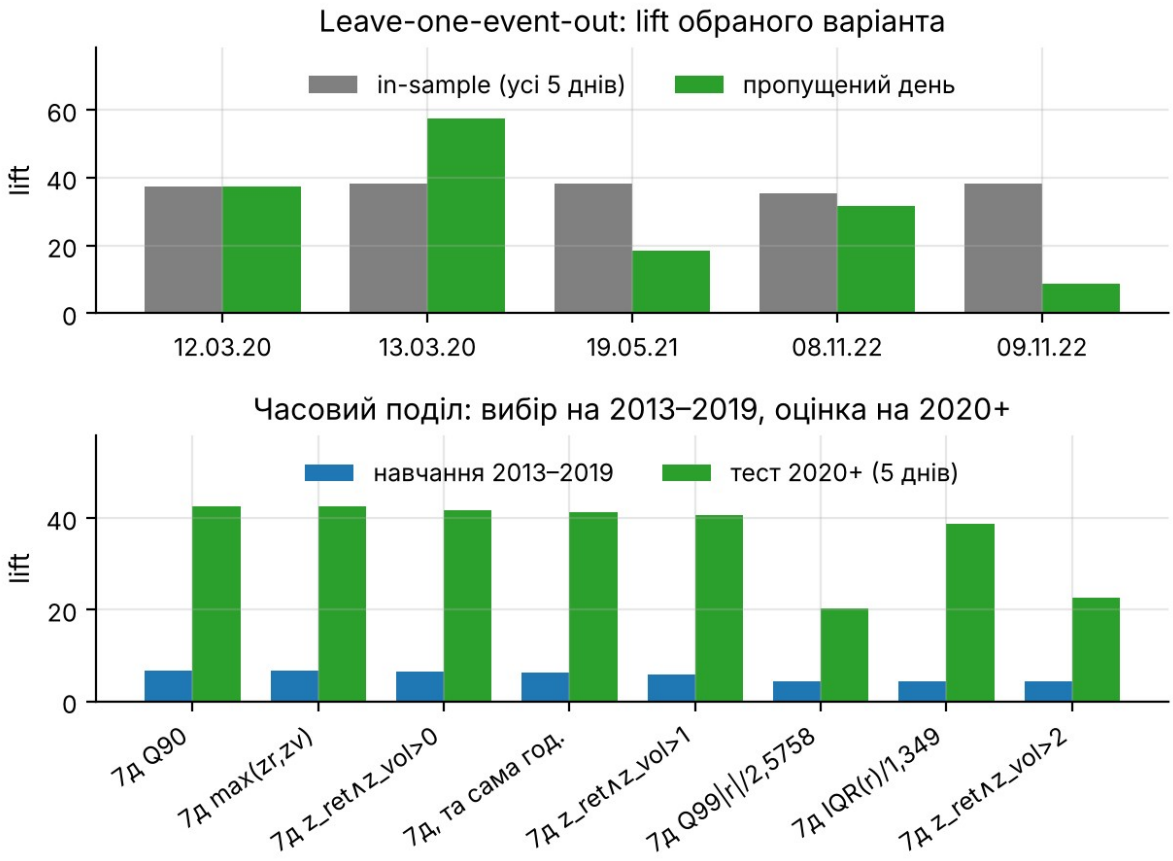


Рисунок 6 – Оцінка поза вибіркою: leave-one-event-out і часовий поділ 2013–2019 / 2020+

7 Збереження результатів

Віконні агрегати записуються parquet-приймачами (append, окремий checkpoint на запит), кожне вікно один раз (дублікатів 0). Оцінки хвилин `foreachBatch` пише у файл з номером мікробатчу, тому повтор після збою перезаписує той самий файл. Для П34 сформовано `pz3_anomalies.csv` – 5 437 аномальних хвилин історії, з них 272 у відтворених періодах. Стан `StreamScorer` живе в драйвері, а не в checkpoint, тому живому потоку

потрібне сховище стану (`applyInPandasWithState`). Каталог `win1h` (133 файли по 2–3 КБ) та початок `pz3_anomalies.csv` – на рисунку 7.

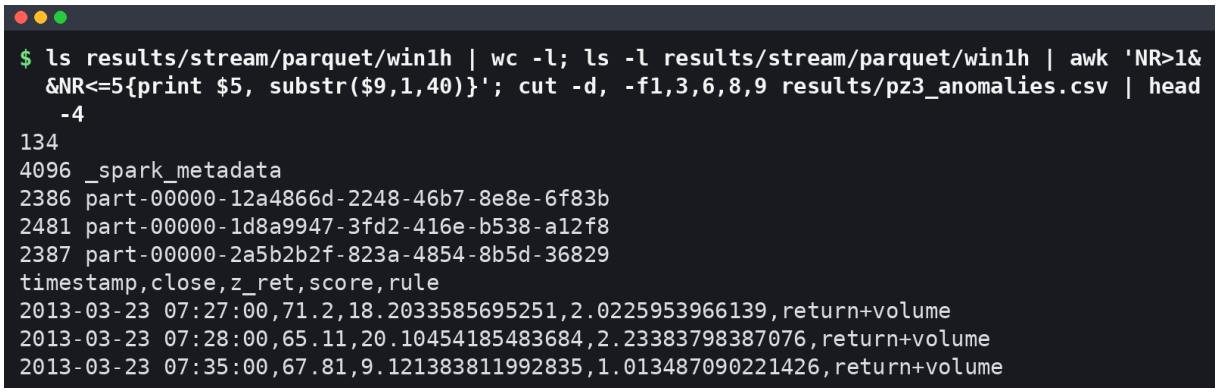


Рисунок 7 – Збережені результати: parquet-приймач вікон 1 год і файл аномалій для ПЗ4

8 Аналіз результатів

8.1 Аномалії в потоці та перевірка формули

Потокову оцінку порівняно з офлайн-оцінкою (таблиця 6).

Таблиця 6 – Аномалії у відтворених періодах: потік і офлайн-оцінка

Період	Оцінено хвилин	Аномалій у потоці	Аномалій офлайн	На добу	Відношення до бази 2018+	Lift (вікна 15 хв)	Lift (вікна 1 год)	max z_ret
2020-03	11 461	166	166	20,86	16,0	8,2	5,6	52,69
2021-05	8 640	29	29	4,83	3,7	2,8	2,4	17,38
2022-11	9 915	77	77	11,18	8,6	5,0	3,7	85,31
Разом	30 016	272	272	–	–	–	–	–

На 30 016 хвилинах max |Δz_ret| = max |Δz_vol| = max |Δsigma| = 0, розбіжностей прапорців 0. Обидві сторони викликають `score()`, тому збіг доводить лише незалежність від мікробатчів і прогріву; формулу окремо перевірено `verify_anomalies_independent.py`: pandas-реалізація для 200 позначених і 200 непозначених хвилин дала 0 розбіжностей, а після заміни наступних 3 000 свічок випадковими z у момент t збіглися в 40 з 40 випадків. База відношення в таблиці 6 – 1,3 аномалії на добу у днях 2018+ без подій.

8.2 Відомі події

У таблиці 7 ранг рахується серед 1 572 дат з позначками (1 293 у 2018+).

Таблиця 7 – Виявлення відомих подій правилом v2 (k = 9)

Дата	Аномалій	Ранг (уся історія)	Ранг (2018+)	Позначених вікон 1 год	max z_ret	Медіана sigma за добу	Lift (хвилини)	Lift поза вибіркою
12.03.2020	82	1	1	11	52,69	0,00104	62,9	37,5
13.03.2020	75	2	2	13	31,26	0,00193	57,5	57,4
19.05.2021	23	27	19	4	17,38	0,00188	17,6	18,4
08.11.2022	59	4	4	8	85,31	0,00048	45,4	31,6
09.11.2022	12	66	51	5	20,61	0,00072	9,2	8,8

Події 19.05.2021 і 09.11.2022 виявлено частково (ранги 27 і 66), бо тижнева база вже містила попередню волатильність: sigma 19.05.2021 у 1,8 раза вища, ніж 12.03.2020, позначено лише 23 хвилини (11:31–14:45 UTC); 09.11.2022 база включала крах попередньої доби, а падіння було поступовим. Правило реагує на раптові хвилинні рухи, а не на багатоденні тренди.

8.3 Закономірності

Три тижні потоку замалі для статистики, тому закономірності оцінено на всій історії тим самим правилом (4 364 з 5 437 позначок – 2018+, рисунок 8).

Частка позначених хвилин у будні 0,113 %, у вихідні 0,052 % (відношення 2,17); за годиною вона зростає з 0,0358 % о 07 UTC до 0,1556 % о 14 UTC – у робочі години американських ринків, хоча частину піку дає база без урахування внутрішньодобової сезонності. Правило нечутливе до знака руху: серед 50 найактивніших дат історії 30 обвалів і 20 зростань, серед 50 дат 2018+ – 25 і 25. Аномалії йдуть сплесками: 71,1 % позначок 2018+ мають іншу позначку в межах 60 хв, тож тривоги доцільно групувати в інциденти.

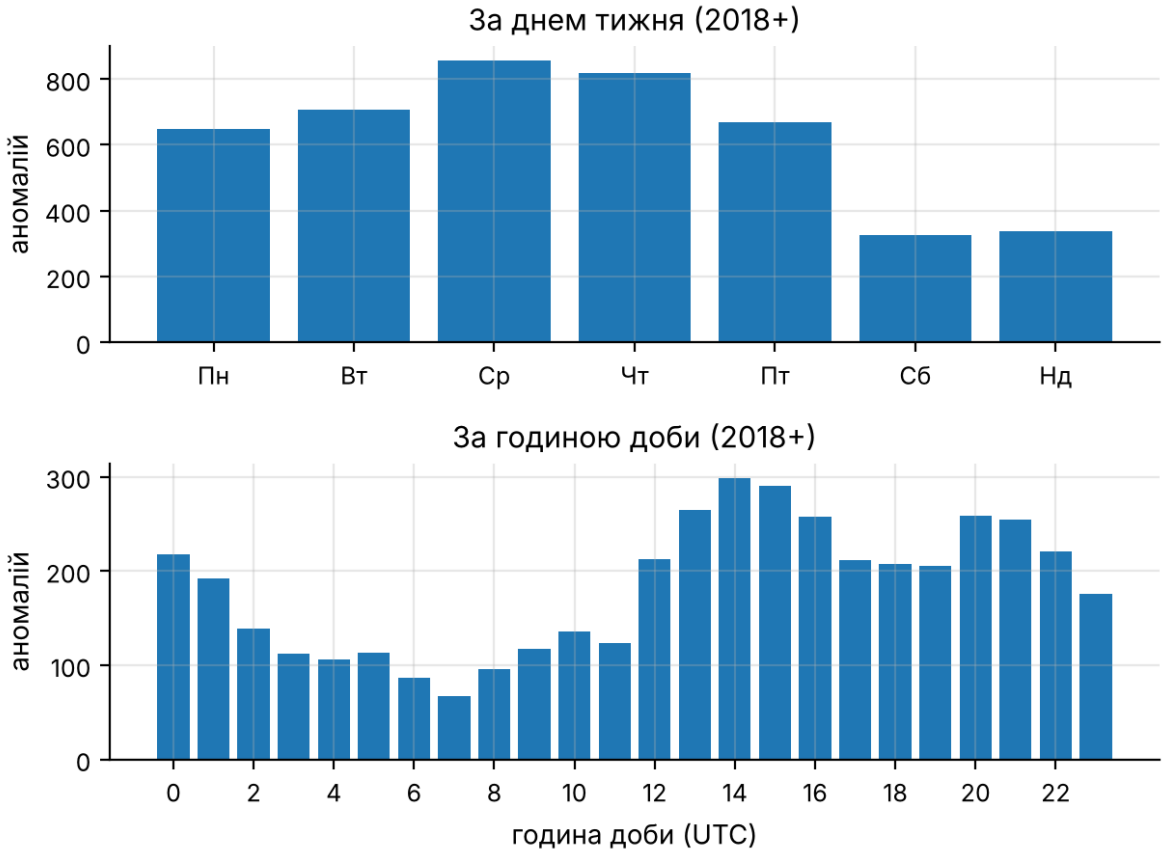


Рисунок 8 – Кількість аномалій за днем тижня і годиною доби UTC, 2018+

8.4 Продуктивність

Метрики `query.recentProgress` наведено в таблиці 8; хост був спільний з іншими завданнями (loadavg 5,11–13,16, у середньому 8,31 на 4 vCPU), тому значення занижені.

Таблиця 8 – Метрики потокових запитів (тригер 10 с, local[2])

Запит	Мікробатчів з даними	Тривалість, середня / медіана / p95 / max, мс	Оброблено рядків/с, середнє	Вхід, рядків/с
anomaly	34	1142,7 / 968,5 / 2384,2 / 2680	919,2	88,9
console_1h	34	2516,8 / 2007,0 / 5006,2 / 6682	428,3	89,0
daily_mem	34	2719,1 / 2363,0 / 6275,5 / 7504	446,2	90,2
win15m_mem	34	2219,7 / 1896,5 / 4287,4 / 5497	488,3	89,1
win1h_mem	34	2307,9 / 2006,0 / 4931,9 / 6996	501,6	89,1
sink_win15m	34	2817,6 / 2535,5 / 4920,9 / 6105	371,4	89,9
sink_win1h	34	2646,7 / 2389,5 / 4357,9 / 5263	380,2	89,6

Усі мікробатчі коротші за тригер (максимум 7 504 мс), а обробка 371,4–919,2 рядків/с перевищує вхід 88,9–90,2 рядків/с. Наскрізна затримка

від CreateTime до початку оцінювання в `foreachBatch` – у середньому 5,651 с, до готового результату – 5,823 с (медіана 5,761 с, p95 10,377 с, максимум 12,445 с); основна частина – очікування тригера (обробка 0,716 с). Офлайн-оцінка всієї історії тривала 68,1 с.

Висновки

Kafka 3.7.0 розгорнуто в режимі KRaft; ідемпотентний producer передав у тему `bigdata-topic` 30 240 свічок трьох періодів обвалів без помилок, а Spark Structured Streaming обробляв їх сімома запитами з декодуванням, фільтрацією й добовою агрегацією.

Віконні OHLC-агрегати 15 хв і 1 год збіглися з `pandas.resample` в усіх 2 016 і 504 вікнах. Водяна позначка 10 хв – запас (невпорядкованість 0), але через її глобальність backlog двох партицій з різними епохами залишив у вікнах 15 хв лише 14 080 з 30 240 рядків. Правило v2 (|r| понад 9 робастних сигм за 7-денною базою Q90 з підтвердженням обсягом) дає 1,38 тривоги на добу; lift днів подій 38,4 на проєктній вибірці за бюджету 0,10 %, 30,7 у leave-one-event-out і 42,5 на 2020+ за порогу з 2013–2019 років.

Потік збігся з офлайн-оцінкою (272 = 272), незалежна перевірка формули розбіжностей не виявила; 5 437 аномалій історії збережено в `pz3_anomalies.csv` для ПЗ4. Аномалії в 2,17 раза частіші в будні, мають пік о 14 UTC, супроводжують і обвали, і зростання (30 і 20 серед топ-50 дат) та йдуть сплесками. Мікробатчі (до 7 504 мс) коротші за тригер, середня затримка – 5,823 с.